CS 3005
Theory of Automata
Lecture 20

Mahzaib Younas

Lecturer, Department of Computer Science

FAST NUCES CFD

Objectives

- **A New Format for FAs**
- **Adding a Pushdown Stack**
- **Defining the PDA**

A New format of FA

- We have learned that all regular languages can be generated by CFGs, and so can some non-regular languages. In other words, the set of CFLs is larger than the set of regular languages.

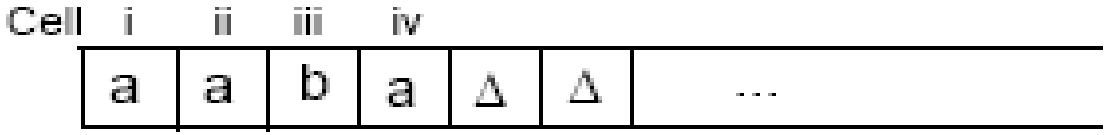
- Since for each regular language, there is an FA that accepts exactly the language, we expect that for each CFL, there should also be at least one machine that accepts exactly the CFL.

- In other words, we would like to have CFL-acceptors (or CFG recognizers) in the same manner as FAs are regular language-acceptors (or recognizers).

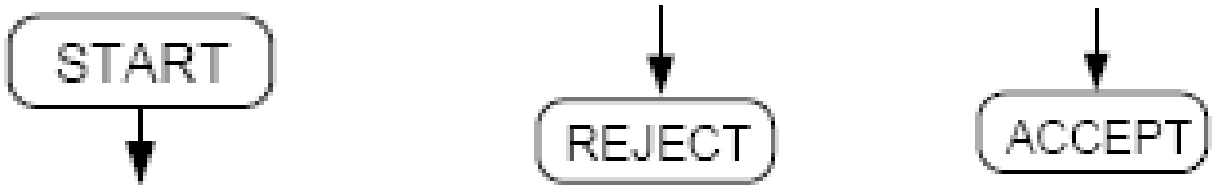
- In this lecture, we shall develop such a new type of machine. In the next lectures, we shall prove that these new machines do indeed correspond to CFLs in the way we desire.

Input Tape

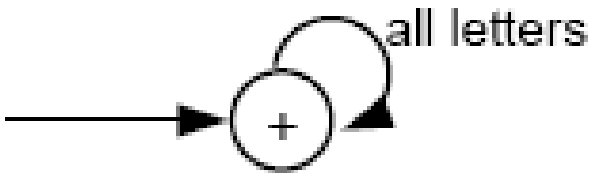
- The INPUT TAPE is part of the machine where the input string is placed. It is infinitely long to accommodate any possible input.
- The locations into which we put the input letters are called **cells**. We name cells with lowercase Roman numerals.
- The character Δ is used to indicate a blank in a TAPE cell.
- We read one letter at a time, from left to right, and never go back to a cell that was read before. When we reach the first blank cell, we stop. We always presume that once the first blank is encountered, the rest of the TAPE is also blank.



Other Symbols



- The START state is like a - state in an FA. We begin the process here, but we read no input letter and go immediately to the next state.
- The ACCEPT state is a dead-end final state + once entered, it cannot be left, such as

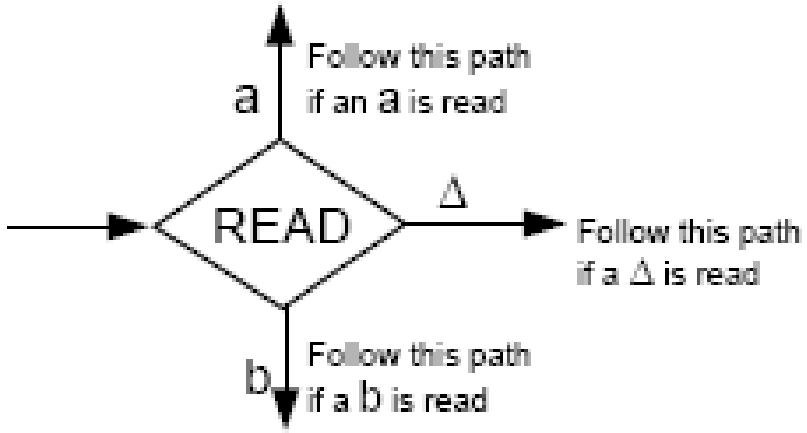


- The REJECT state is a dead-end state that is not final, such as

- The ACCEPT and REJECT states are called **halt** states. Halt states cannot be traversed (i.e., cannot be passed through like a final state in an FA).

READ States

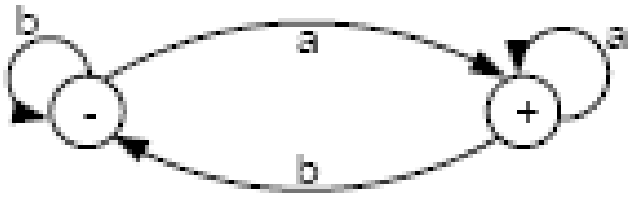
- READ states are depicted as diamond-shaped boxes as shown below:

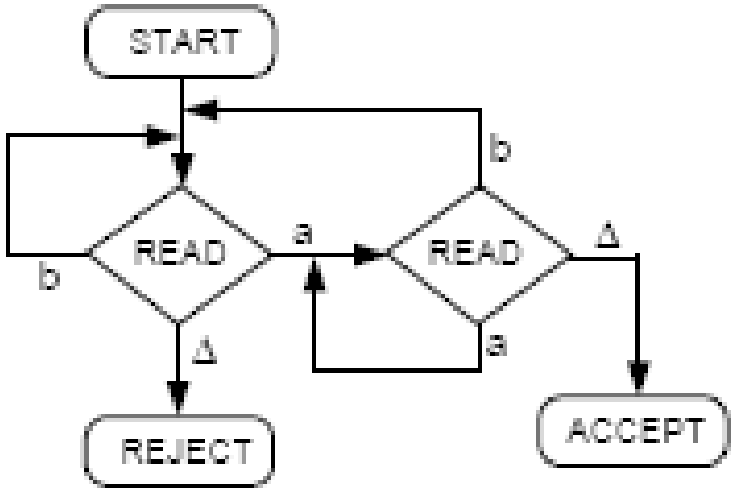


- When Δ is read from the TAPE, it means that we are out of input letters. We are then finished processing the input string. Hence, the Δ-edge will lead to ACCEPT or REJECT.

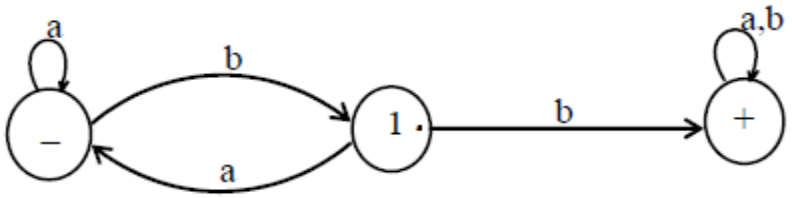
Example

- Below is an FA that accepts all words ending with letter a, and its equivalent new picture.
- The Δ edge should not be confused with Λ-labeled edge. The Δ-edges start only from READ boxes and lead to halt states.

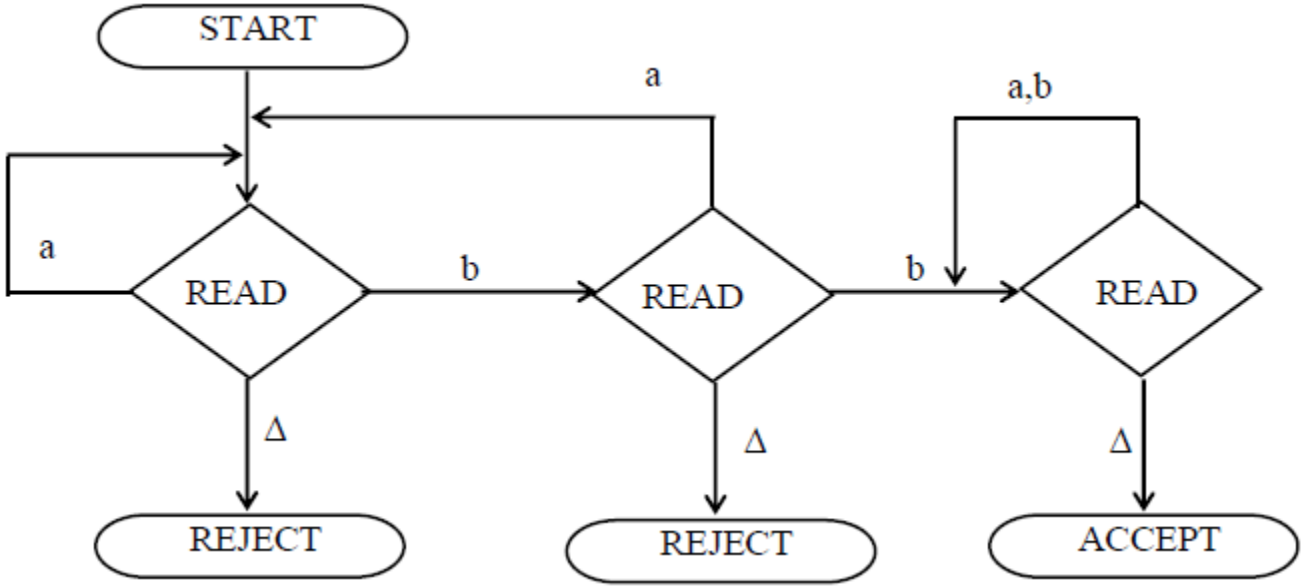




PDA



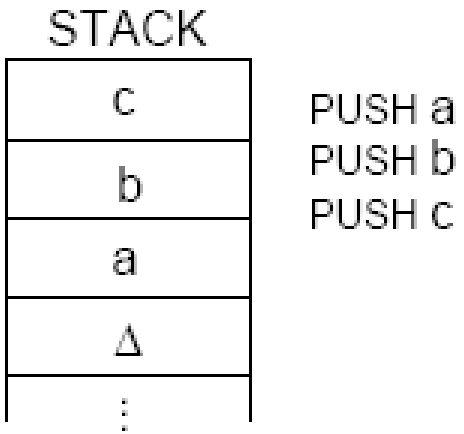
- The above FA accepts the language expressed by (a+b)*bb(a+b)*



Adding a Pushdown Stack

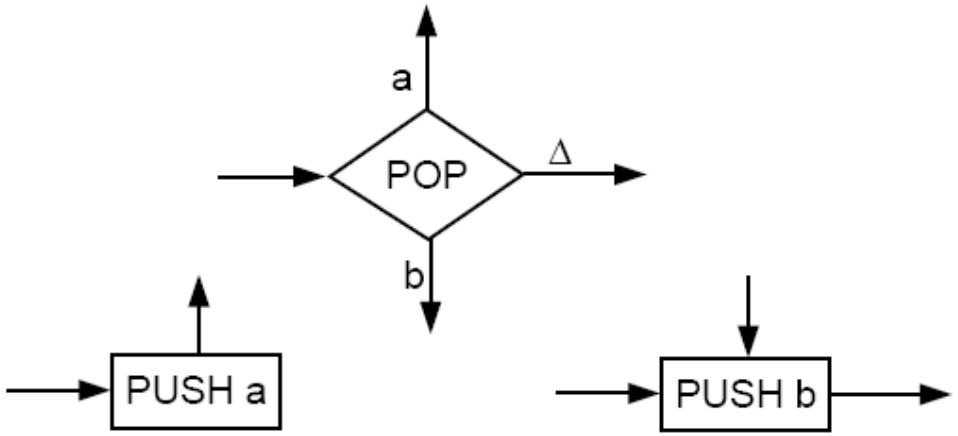
- We bothered to construct new pictures for FAs so that it is now easier to make an addition to the machine.
- We would like to add a **PUSHDOWN STACK** to our machine. A PUSHDOWN STACK is a place where input letters can be stored until we want to refer to them again.
- Before the machine begins to process an input string, the STACK is presumed to be empty.
- The operation PUSH adds a new letter to the top of the STACK. All the other letters are pushed down accordingly.
- The operation POP takes a letter out of the STACK. The rest of the letters are moved up one location each accordingly.

- For example, if we perform operations PUSH a, PUSH b, and PUSH c, then the stack will look like the following:



- Popping an empty STACK, like reading an empty TAPE, gives us the blank character Δ.

- We can add a PUSHDOWN STACK and the operations PUSH and POP to our machine by including the following symbols:

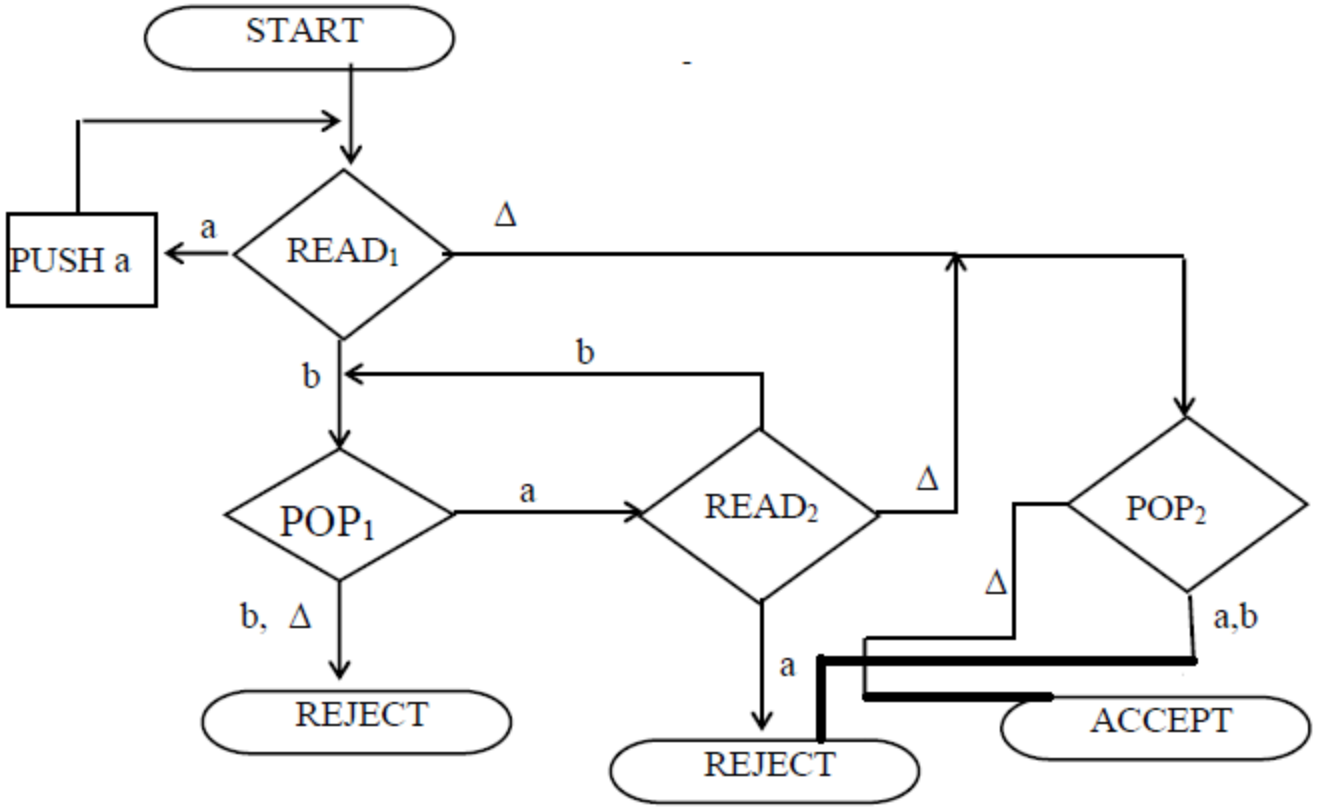


- The edges coming out of a POP state are labeled in the same way as the edges coming out of a READ state.
- Branching can occur at POP states but not at PUSH states: We can leave PUSH states only by the one indicated route, although we can enter a PUSH state from any direction.

- When FAs are equipped with a STACK and POP and PUSH states, we call them **pushdown automata** or **PDAs**.

- The notion of a PUSHDOWN STACK as a data structure has been around for a long time. However, when we incorporate this memory structure into an FA, its language-recognizing capabilities are increased considerably.

- Consider this PDA:



- What language is accepted by this machine?

- Let's see the machine in operation on the input string ***aaabbb.***
- At the beginning the INPUT TAPE contains (from left to right) the letters a, a, a, b, b, b, Δ, Δ, Δ…; while the STACK is empty, that is, it contains (from top down) only the blank characters Δ, Δ, …
- After reading the first a, the remaining input letters to be read in the TAPE are a, a, b, b, b, Δ, … and the STACK looks like (from top down) a, Δ, …
- Similarly, after reading 3 letters a, we have

  TAPE = b, b, b, Δ, … STACK = a, a, a, Δ, …
- After reading the first b, STACK pops an a, so

  TAPE = b, b, Δ,… STACK = a, a, Δ, …

- Similarly, after reading the last b, STACK pops the last a, and we have

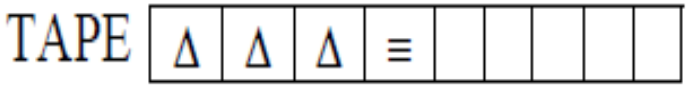
  TAPE = Δ, Δ, … STACK = Δ, Δ, …
- When we encounter the blank character Δ from the INPUT TAPE, the STACK pops a blank , and we are lead to the ACCEPT state.
- Hence, the string ***aaabbb*** is accepted by this machine.

- In fact, the language accepted by this machine is the **non-regular** language

  $\{a^n b^n \; n = 0, 1, 2, 3, \ldots\}$

- This machine can accept this non-regular language because it has a memory unit (the STACK) to keep track of how many a's were read at the beginning. Since FAs do not have such memory, they are not able to accept this language.

- Since the null string is like a blank character, so to determine how the null string is accepted, it can be placed in the TAPE as shown below



- Reading Δ at state READ1 leads to POP2 state and POP2 state contains only Δ, hence it leads to ACCEPT state and the null string is accepted.

- The process of running the string aaabbb can also be expressed in the following table

STATE	STACK	TAPE
START	Δ …	aaabbbΔ …
$READ_1$	Δ …	aaabbbΔ …
PUSH a	aΔ …	aaabbbΔ …
$READ_1$	aΔ …	aaabbbΔ …
PUSH a	aaΔ …	aaabbbΔ …
$READ_1$	aaΔ …	aaabbbΔ …
PUSH a	aaaΔ …	aaabbbΔ …
$READ_1$	aaaΔ …	aaabbbΔ …
POP_1	aaΔ …	aaabbbΔ …

STATE	STACK	TAPE
$READ_1$	aaΔ …	aaabbbΔ …
POP_1	aΔ …	aaabbbΔ …
$READ_2$	aΔ …	aaabbbΔ …
POP_1	Δ …	aaabbbΔ …
$READ_2$	Δ …	aaabbbΔ …
POP_2	Δ …	aaabbbΔ …
ACCEPT	Δ …	aaabbbΔ …

Remarks

- We need not restrict ourselves to using the same alphabet for input strings and for the STACK.
- In the example above, we could have read an a form the TAPE and then pushed an X into the STACK; that is, we let the ***X***'s count the number of a's.

- In this case, the READ state must provide branches for a, b, or Δ. The POP state must provide branches for X or Δ.

- Therefore, when we define PDAs later on, we shall require the specification of the TAPE alphabet ∑ and the STACK alphabet Γ, which may be different.